Data Streaming - Message Brokers

This train will focus on general message broker concepts, service models, and event streaming, ensuring students understand the broader context before diving into Kafka-specific implementations.

Learning Objectives

- Fully explain a message broker service and its role in real-time data processing.

Message Broker Services - Publish Subscription Services

Modern streaming data systems often rely on a core component called a message broker. This broker acts as a central hub, allowing 'producers' to continuously send new data, and 'consumers' to instantly receive and process that data in real-time. The concept of a high-throughput, distributed message broker gained prominence with LinkedIn's development of Kafka in 2008. Kafka was designed to handle massive volumes of data efficiently. It was later open-sourced as Apache Kafka, becoming a widely adopted standard. Confluent, founded by the original creators of Kafka, further developed and commercialized the technology, contributing significantly to its ecosystem and adoption. While cloud providers now offer managed streaming services inspired by Kafka's principles, Apache Kafka itself remains a foundational, open-source technology, often deployed directly or through distributions like Confluent's.

The figure below details the structure of a basic pub/sub message broker, with producers producing messages to exchanges and queues, and from where they are, in turn, consumed by consumers.

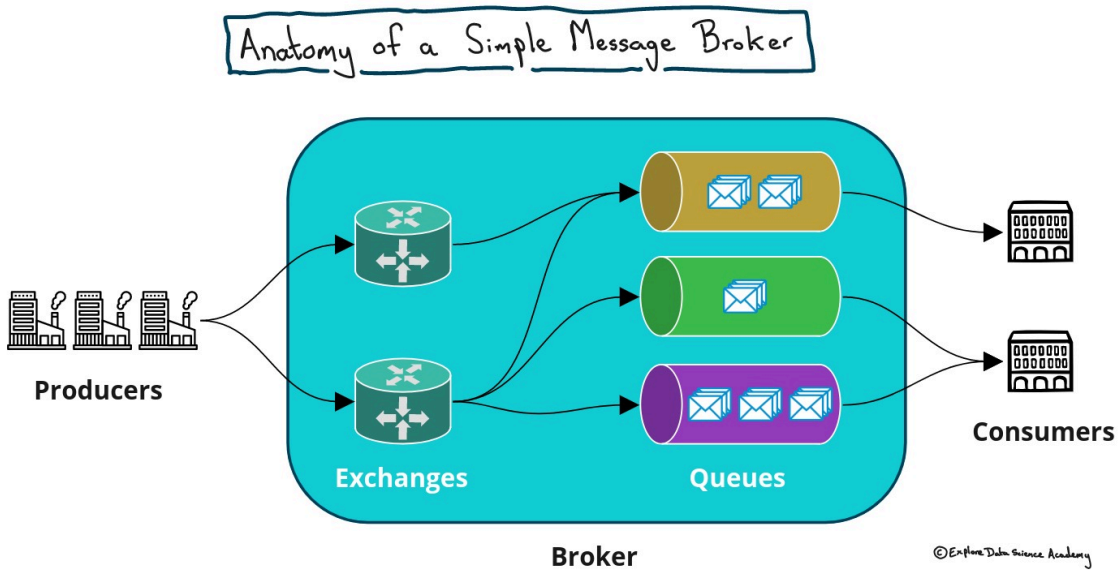


Figure 1: The basic components of a message broker, with producers pushing events to exchanges which, in turn, route the events to queues to await processing via consumers.

Core components of message brokers that allow them to power streaming systems:

A - Append-only log

At the base of these technologies is the *append-only log*. A log is effectively a sequence of binary records. They can be in a multitude of formats: JSON, Apache Avro, and more. The append-only log allows for the addition of records only to the end of the log, meaning that you cannot insert arbitrarily or remove any records. Each record in the log will have a log number, with newer items having higher numbers than older items.

A consequence of this is that when a record is read by a consumer it cannot delete the record, but rather the record is retained and can be read again by other consumers. As such, the consumer keeps track of which records have been read while the broker itself is ignorant of this detail (both of which records have been read and by how many consumers). This allows these systems to process an inordinate amount of records with low latency.

B - Retention

One thing which should now be clear is that for a high-velocity data stream the append-only log will rapidly grow large, and while these messages are typically quite small, we cannot store their records indefinitely. Thus, we have to remove them at some point.

Most of the current message broker services have a retention period of between 24 hours and seven days (which can be extended arbitrarily, provided sufficient storage exists). This extended retention period can be of great benefit when an error has occurred somewhere on the processing pipeline and we have to rerun our ingestion pipelines, or alternatively, the stream volume increased unexpectedly and a backlog started to accumulate before we noticed or could remediate it. This gives us time to fix any errors in our production system. However, it means we have to pay more for the additional storage associated with the log.

C - Scaling out

Streaming applications can theoretically scale infinitely, assuming that more and more IoT sensors can be added to a system or the sensors become increasingly complex and sophisticated in the data they send back to the streaming application. For this reason, streaming systems are built to handle petabytes of data – a feat that is achieved through scaling horizontally.

D - Sharding

One of the methods streaming services use to deal with an increase in reads and writes is through a process called *sharding*. Sharding occurs when we split our append-only log into several horizontal partitions, with each of these being placed on a separate broker instance.

To determine which shard to write to, we create a key for each incoming record. This is normally a hash code of the incoming record. The value of the hash is used to determine which shard should process it, as each shard is responsible for a portion of the hash code value range. Hash values can be sampled from a random distribution, which will ensure uniform distribution of incoming data among shards. Alternatively, hash values can be domain-specific, being a field (or combination of fields) within

the dataset. Similarly to databases, which have a domain-specific index, having domain-specific hashes can have performance benefits (for example, having data aggregated on one shard increases the speed of aggregations).

The number of shards in the system will depend on the latency and number of IOPS (input/output operations per second) required from the streaming application. Streaming tools place a limit on the number of reads from a particular shard, which can sometimes lead to the accumulation and queuing of read requests. In this event, replication enables reading from more than one shard, reducing the load on any one shard.

E - Replication

Another method for increasing the read and write speed of a message broker is through the use of replication. Having multiple copies of a log allows for a decrease in latency since any one of the copies can be used to read from. Since streaming tools place a limit on the read from a particular shard, which can sometimes lead to the accumulation and queuing of read requests, this allows reading from more than one shard if this happens, lowering the load on any one shard.

Additionally, replication has the benefit of making the system more resilient to failure, with second and third copies of a log shard being maintained when one copy fails.

Now that you are familiar with the basics of streaming as well as the architecture of a message broker, let's get into the detail of some of the current message brokers and related offerings available.

Reflect:
This section explored the key mechanisms that enable message brokers to effectively handle the demands of streaming data. The concept of an append-only log was introduced, highlighting its role in preserving data order and facilitating efficient consumption. Retention policies were discussed as a means of balancing storage needs with data availability for recovery and reprocessing. Additionally, the importance of scaling out and sharding was emphasized as strategies for managing large data volumes and ensuring high throughput.

Finally, the role of replication in enhancing fault tolerance and read performance was explained. These concepts collectively illustrate the sophisticated design of message brokers and their ability to provide a reliable and scalable foundation for real-time data processing in streaming applications.

Conclusion

In this train, we've crossed the deep river of data streaming. Streaming and data streaming is a massive topic, and we can probably build a one-year course on the subject alone. The aim of this train, however, was to provide a theoretical overview of how a streaming application works. While batch processing is still very prevalent in most industries and domains, we are increasingly moving to a place where streaming and seeing the world with a real-time perspective are becoming important to ensure competitiveness.

Key Takeaways

- **Message Broker Function:** A central hub enabling real-time data flow from producers to consumers, crucial for modern streaming architectures.
- **Append-Only Log:** The core mechanism, an ordered, immutable sequence of records that allows consumers to read data without deleting it, supporting high throughput.
- **Data Retention:** Messages are stored for a configurable period, providing resilience for reprocessing data and recovering from pipeline errors.
- **Scalability via Sharding:** Message brokers achieve high scalability by sharding (partitioning) logs across multiple broker instances for parallel processing.
- **Resilience through Replication:** Multiple copies of logs (replication) enhance fault tolerance and improve read/write speeds by distributing the load.